Machine Learning for Signal Processing

[5LSL0]

Rik Vullings
Ruud van Sloun
Hans van Gorp
Louis van Harten
Tristan Stevens
Oisín Nolan
Simon Penninga

Assignment 2: CNNs

Academic Year 2025-2026

Use of generative AI

You are allowed to use generative AI, specifically to create content (like images, (source) code, video, text or any other kind) and writing. However, you remain solely responsible for the answers you provide and you remain solely responsible for your own learning process. In the written exam, use of generative AI is not permitted.

Grading

The final exercise in each assignment is graded; in total these exercises contribute 20% of your final grade. The other exercises are not graded, but the code and/or understanding you build along the way is typically required to solve the final exercise.

Assignments that require you to code in python are denoted with a icon. You may code in any IDE you like. For your own benefit, keep the code clean, legible, and include lots of comments; code that you create for this assignment can be repurposed for later assignments.

Submission format

Code needs to be collated into three standalone files: `exercise_1.py`, `exercise_2.py` and `exercise_3.py`. Which should create all the figures and tabels. We suggest you include a command line argument `--load` in your scripts, which loads your saved results (`.tar`-files) to generate the figures without retraining everything. Make sure the script produces all to-be-loaded files when called without this argument.

Answers that are not code should be provided as a `.tex`-document. Make sure that the answer to each question is clearly labeled using a `\subsection{1.1a}` command. We will parse everything that is placed between e.g. `\subsection{1.2b}` and `\subsection{1.2c}` as being part of the answer to question 1.2b.

Compress your answers, i.e. the `.py`, `.tex`, and `.tar`, into a single `.zip` archive, with the name `assignment_2_[studentnumber]_[studentnumber].zip`. You are allowed to work in groups of two. Note that only one student needs to submit to canvas. Make sure that we know exactly who is part of your group.

Workload

Assignment 2 consists of three exercises. Only the last one is graded, but completing the first two is highly recommended, as doing so will make your life much easier when working on the last exercise. There are a number of optional questions that allow you to earn bonus points. The expected workload for this assignment is **8 hours**.

Training data and a LaTeX template for your answers can be found on Canvas.

Exercise 2.1: From MLP to CNN

In this exercise, you will explore the connection between MLPs and CNNs, and explore their differences in properties.

(a) A linear layer in an MLP connecting an input of length 5 to a hidden dimension of length 10 can be represented by a weight matrix $W \in \mathbb{R}^{10\times5}$ and a bias vector $b \in \mathbb{R}^{10}$, containing 60 parameters in total. This same weight matrix may also represent a set of 2 convolutional filters of length 3. In the latter case, how many parameters do we have?

(b) What does the matrix W look like in the above case? Use a different variable for each parameter. Assume zero-padding for consistent feature map sizes.

(c) 🐍 In Pytorch, initialize a `torch.nn.Conv1d` layer to match the situation described above. Also initialize a linear layer with 5 inputs and 10 outputs. Copy the weights from the convolutional layer to the linear layer, as you described in (1b). Now generate a random vector of length 5 and run it through both the linear and the convolutional layer, and print the output for both. Are they the same? If not, how do they differ?

(d) 🐍 Repeat exercise 1(a)-(d) from assignment 1, replacing the denoising MLP with a small CNN.

(e) 🐍 Again plot the outputs of your untrained network together with the results from A1:1(e). Plot these results in a figure with three rows, where the first row shows the inputs, the second row shows the untrained MLP outputs, and the third row shows the untrained CNN outputs.

Check your understanding:

- The last layer of your MLP had 1024 outputs; how many feature maps should the last layer of your CNN have?
- When using 3x3 convolutions, what padding should we use to retain the original size?
- What is the qualitative difference between the output values of your untrained MLP and your untrained CNN? What causes this difference?

(f) 🐍 Now train your CNN at least three times with three different learning rates and plot the validation loss curves. Also load and plot the validation loss graph from your best MLP-based model. Remember to add a legend with descriptive labels.

(g) 🐍 Run the CNN with the best validation loss on your test set, and plot a comparison of the input and the output images.

(h) 🐍 Pick an image from the test set and create two 28x28 sub-images from it: one where the top and leftmost 4 pixels are cut off, and one where the bottom and rightmost 4 pixels are cut off. Now run your model on both sub-images. Crop the output of both images to 24x24 such that they correspond to the same input pixels again. Plot a figure with three columns, showing both output images and their difference image.

Check your understanding:

- CNNs are supposed to be perfectly equivariant, which means the difference image in 1(h) should be all exactly zero. Is it? If not, where are these differences coming from?

Exercise 2.2: Audio Source Simulation

In this exercise, you will build a system that triangulates audio sources from a set of microphones, for example to automatically detect whose phone is ringing during a lecture. We'll use a simple physics model to simulate audio delays, and then use the simulated data to train a network. First, let's build a very simple simulator:

(a) Given a microphone at coordinates (x_{mic}, y_{mic}) and a sound source at (x_s, y_s), what is the time delay of the recorded signal compared to the source? Assume a homogeneous sound speed $c = 340$ *m/s*.

(b) Implement a function `audio_simulator(input, source_loc, mic_locs)`, which takes a mono waveform as input, calculates the relative time delays to each microphone in the list `mic_locs` and generates a set of waveforms that this microphone array might record given the input waveform. Assume a sampling rate of 48kHz. For efficiency, round the relative time delays to the nearest sample.

Important! *A simple implementation of time delays would be to prepend zeros, but this assumes knowledge of when audio sources start emitting. We typically have no knowledge or control over when sources start, meaning this simulator would give us information we would not usually have (as perfect zeros are distinct from the background noise a microphone will pick up). Instead, crop the source signals such that the relative time delays in the simulated mic traces are correct.*

Now load an audio file to test your simulator. Either use one of the sound files from the zip on Canvas, or simply record something with your own microphone.

(c) Load your audio file using `torchaudio.load` or `torchcodec.decoders`; if your file was stereo, average over the channel dimension to make it mono. Define two microphones at (-8 cm, 0) and (8 cm, 0) (approximately matching the distance between your ears), set the source at (200, 100 cm) and save the resulting audio. Plot a segment of both output channels; zoom such that the time delay difference is visible and label your axes.

Play the result back on headphones. Does the sound seem to come from where you expect? If not, debug your simulator code until it does.

Also test your code with 3 and 4 microphones; how could you verify the correctness now?

We will use **4 microphones**, at (-45 cm, 0), (−15 cm, 0), (15 cm, 0) and (45 cm, 0).

(d) Explain why we need more than two microphones for source localization. Optionally, use a figure to clarify.

(e) What is the maximum possible time offset with the microphones positioned as above? With how many samples does that correspond?

(f) How should that influence your choice of receptive field when designing a network to process this data?

(g) Write a Pytorch dataloader to generate data samples with your simulator, simulating a mix of sounds coming from multiple locations. Generate two figures showing a data sample for 1 and for 2 sources.

Your dataloader should:

- Load all sound files in a given folder and cache them in memory (because loading from disk is slow).
- Have the following initialization parameters:
  - `data_folder` specifying where to load the data
  - `t` in milliseconds (i.e. how long the returned samples should be)
  - `mic_locs`, a list of microphone locations in meters
  - `n_sources_min`, `n_sources_max`, specifying the range for the random number of simulated sources
  - `x_loc_min`, `x_loc_max`, `y_loc_min`, `y_loc_max`, specifying the ranges for the random source locations
- Randomly choose between `n_sources_min` and `n_sources_max` sounds to simulate in random locations. Crop any zeros your simulator may have added, then choose a random segment of `t` milliseconds for each simulated sound, and add them together.
- The resulting waveforms should be a tensor of shape $[n_{mics},\ samples]$ (where `samples` corresponds to `t` milliseconds). Also return a tensor with each individual simulated sound segment (shape: $[n_{sources},\ n_{mics},\ samples]$), and the location for each source (shape: $[n_{sources},\ 2]$).

- Before moving on, make sure you have verified the correctness of the samples coming out of your dataloader, both for single-source and for multi-source samples. By selecting two of the three microphone outputs, you can construct a stereo sound file. Try this for two different combinations of microphones and play them on headphones. How do the two files differ? Does that match your expectations?

Exercise 2.3: Audio Source Localization and Isolation [GRADED]

This is the graded exercise. You can work together with one other student. If you do, make one shared submission in Canvas, including both of your names and student numbers.

In the previous exercise, we made a simulator to create data samples for locating different audio sources. By using a CNN, we can train on short snippets of audio, and at test time, stream microphone input directly into the network without having to chop it up into short snippets. Then we can show the network predictions in real time.

(a) How do strided downsampling layers in a CNN affect the receptive field?

(b) A CNN without strided downsampling layers is equivariant to the symmetry group of all integer-pixel translations. If we use two strided downsampling layers in our CNN, what symmetry group is the network equivariant to?

Let's start easy: locating a single source. Initialize dataloaders for the training and validation sets, with `n_sources_min` = `n_sources_max` = 1, $x_s \in$ [-200, 200]cm, $y_s \in$ [25, 200]cm and $t = 0.1$ s.

(c) The simulator quantizes the delays to integers, meaning nearby locations will result in the same simulation results. Create a figure showing which areas result in the same simulation results for (x, y) source locations (0, 50), (-100, 50), (0, 150) and (100, 150)cm. Explain your process.

(d) Design a 1D neural network with two outputs: the estimated x- and y-coordinate of the source. Explain your design choices.

- Implement and train your model. Plot both the loss graph and the average validation accuracy in centimeters over time.
- (optional:) You are encouraged to iterate on your design and change things if you realize your design choices were sub-optimal. Explain the changes you made to your original design, the reasoning behind those changes, and whether this resulted in improvement.

(e) Calculate the receptive field of your model.

- Verify this result experimentally; explain your process and include a figure.
- Does the theoretical result match the receptive field that you find in practice?

(f) Once you're done developing your model with the training and validation set, run the model on your test set. Explain your process of evaluating the performance and report your results. Include a representative figure visualizing the performance. Also comment on how the test-set performance compares to the validation set performance, and how this compares to the quantization ambiguity found in (c).

Once the single-source case is working, we'll make the problem a bit more difficult for the network. Instead of just one source, there may be up to three sources playing at the same time.

(g) Design and train a 1D neural network that can locate between 1 and 3 sources simultaneously. For example, you can use a network where the the coordinates of the closest source are estimated by the first two outputs, the second source by the next two, and so forth, with one final network output estimating how many sources are active. Again, explain your design choices and visualize the training process in a figure.

- (optional:) You are encouraged to iterate on your design and change things if you realize your design choices were sub-optimal. Explain the changes you made to your original design, the reasoning behind those changes, and whether this resulted in improvement.

(h) If we order our outputs by distance from the microphones as suggested above, what may happen if two sources in a training sample are at similar distances? Is that a problem, and if so, how could we fix it?

(i) Once you're done developing your model, run the model on your test set. Explain your process of evaluating the performance in different scenarios and report your results. Include representative figures visualizing the performance.

(j) Simulate the full recording of both bird sounds in the test set as sources located at (0, 50)cm and (100, 150)cm respectively. Plot the estimated location error over time for both. Does the location error correlate with the sound volume of the recordings at different times? Comment on your findings.

Congratulations, you're done with the assignment! Unless...

(k) **(optional, bonus question:)** The "cocktail party problem" is when we try to focus on a sound source while tuning out other sources (e.g. in a lecture, when you're trying to listen but students behind you are talking). Design and train a network that tackles this problem: given a combination of multiple sources, the network should recreate the microphone responses as if only the closest source was active. Explain your design decisions and report the performance of your model. Also create sound files of your network output and listen to them on headphones; qualitatively, did your model do a good job? What are its strengths and weaknesses? If you tried multiple versions (e.g. with different loss functions), is there a qualitative difference?

- (optional:) Modify and train your network to take the source location it should focus on as an additional input, instead of focusing on the closest source. Explain how you incorporate this input in your network. Again listen to the results on headphones. Does this new network perform better, or worse than the network trained to focus on the nearest source? You are encouraged to iterate on your design and change things if you realize your design choices were sub-optimal.

Submission format (repeat of info from above)

Code needs to be collated into three standalone files: `exercise_1.py`, `exercise_2.py` and `exercise_3.py`. Which should create all the figures and tabels. We suggest you include a command line argument `--load` in your scripts, which loads your saved results (`.tar`-files) to generate the figures without retraining everything. Make sure the script produces all to-be-loaded files when called without this argument.

Answers that are not code should be provided as a `.tex`-document. Make sure that the answer to each question is clearly labeled using a `\subsection{1.1a}` command. We will parse everything that is placed between e.g. `\subsection{1.2b}` and `\subsection{1.2c}` as being part of the answer to question 1.2b.

Compress your answers, i.e. the `.py`, `.tex`, and `.tar`, into a single `.zip` archive, with the name `assignment_2_[studentnumber]_[studentnumber].zip`. You are allowed to work in groups of two. Note that only one student needs to submit to canvas. Make sure that we know exactly who is part of your group.